Security Testing

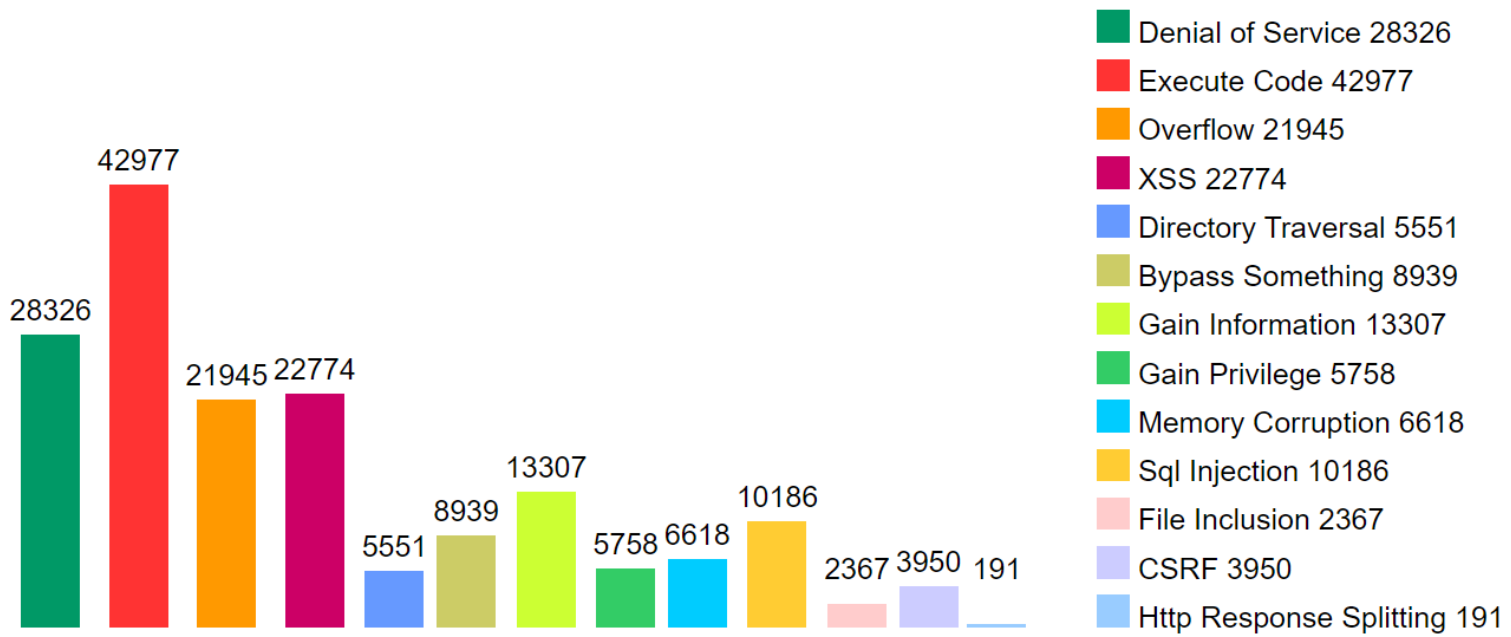
CWE and OSWAP Top 10

(part 2)

Outline

- CWE Top-10
- **OWASP Top-10 vulnerabilities (part 2)**

Vulnerability frequency



CVE vulnerabilities published by error types from 1999
CVE = Common Vulnerabilities and Exposures

https://www.cvedetails.com/vulnerabilities-by-types.php

Classification and groups of bugs / errors

Several approaches to classify software error types exist and they are based on several classification criteria:

- By severity: it refers to the technical impact that the bugs/errors have on the system. It measures how severe the issue is and how critical it is to fix it. In general, rated as: high, medium, low ..
- By priority: it refers to how quickly the issue should be resolved, based on the business impact and the project timeline. In general, rated as: high, medium, low ..
- By nature: it refers to the technical characteristics of the bugs/errors (e.g., functional or security bugs, design or code specific bugs, technology-related bugs)

Classification and groups of bugs / errors

Several approaches to classify software error types exist and they are based on several classification criteria:

- By severit em. It measur as: high, mec
- By priorit business i ..
- By nature onal or security bugs, design or code specific bugs, technology-related bugs)

These criteria are typically applied when:

- a software failure is observed, i.e., our software is affected by a vulnerability
- we observed one/more (code) weakness in our software that could potentially lead to software failures

in such cases, we have to consider severity, priority, and nature of such issues to make decisions…

Why the classification is so relevant?

The classification can be the first step of investigation for:

a) conducting an effective testing activity
   - know what to search while testing
   - prioritize the issues to search
   - evaluate the effectiveness of various testing approaches (e.g., code vs error coverage)

b) improving the development process
   - know solutions (patterns,. guidelines) to help developers in improving their software, thus avoiding to introduce, e.g., weaknesses

c) driving bug-fixing
   - drive the bug fixing: select the developers, locate the bug, understand how to fix it
   - prioritize the issues to solve

Classification and groups of bugs / errors

Can we identify groups of "error" types?

Yes! Taxonomies and classifications exist, e.g.,

[1] CWE Top-25 and Top-10 most dangerous software weaknesses
[2] 24 Deadly Sins of Software Security: Programming Flaws and How to Fix Them
[3] Classification of Software Security Errors
[4] A Taxonomy of Software Security Defects for SST
[5] *Data Type Bugs Taxonomy: Integer Overflow, Juggling, and Pointer Arithmetics in Spotlight.
[6] *An empirical validation of a web fault taxonomy and its usage for web testing
[7] *Machine Learning-based Software Bug Classification through Mining Open-Source Repositories.
[8] *The Python/C API: Evolution, Usage Statistics, and Bug Patterns.
[9] OWASP Top-10

- ...

[1] https://cwe.mitre.org/top25/index.html
[2] M. Howard et al. 24 Deadly Sins of Software Security: Programming Flaws and How to Fix Them. McGraw-Hill Osborne Media, July 2009.
[3] K.Tsipenyuk et al. Seven Pernicious Kingdoms: A Taxonomy of Software Security Errors. IEEE Security and Privacy https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf
[4] Hui et al. A Taxonomy of Software Security Defects for SST. ICISS 2010
[5] I. Bojanova et al. Data Type Bugs Taxonomy: Integer Overflow, Juggling, and Pointer Arithmetics in Spotlight. STC 2022
[6] A.Marchetto et al. An empirical validation of a web fault taxonomy and its usage for web testing. JWE 2009
[7] J.P.Meher et al. Machine Learning-based Software Bug Classification through Mining Open Source Repositories. OCIT 2023
[8] The Python/C API: Evolution, Usage Statistics, and Bug Patterns. SANER 2020
[9] https://owasp.org/Top10/

* Not strictly related to security

CWE Top-10 Weaknesses (recall)

1. Use-After-Free (CWE-416)
2. Heap-based Buffer Overflow (CWE-122)
3. Out-of-bounds Write (CWE-787)
4. Improper Input Validation (CWE-20)
5. Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78)
6. Deserialization of Untrusted Data (CWE-502)
7. Server-Side Request Forgery (SSRF) (CWE-918)
8. Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843)
9. Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22)
10. Missing Authentication for Critical Function (CWE-306)

From the analysis of the known exploited vulnerabilities (KEV)
https://cwe.mitre.org/top25/archive/2023/2023_kev_list.html

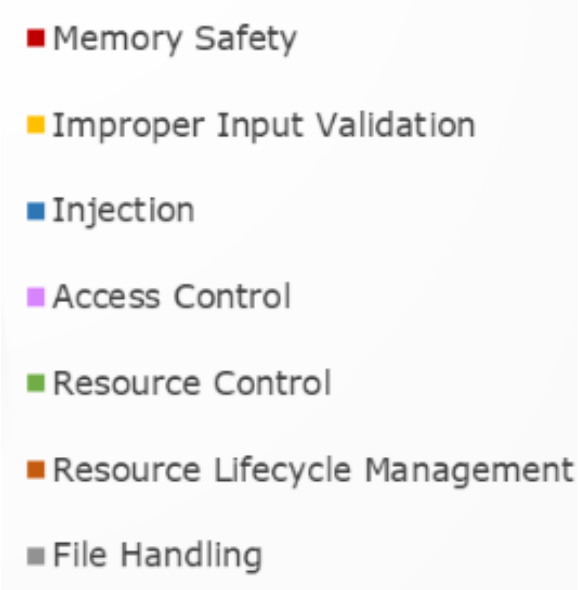
- Code weaknesses
- Most prevalent and severe
- Analysis based on the exploited vulnerabilities

CWE Top-10 Weaknesses (recall)

1. Use-After-Free (CWE-416)
2. Heap-based Buffer Overflow (CWE-122)
3. Out-of-bounds Write (CWE-787)
4. Improper Input Validation (CWE-20)
5. Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78)
6. Deserialization of Untrusted Data (CWE-502)
7. Server-Side Request Forgery (SSRF) (CWE-918)
8. Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843)
9. Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22)
10. Missing Authentication for Critical Function (CWE-306)

From the analysis of the known exploited vulnerabilities (KEV)
https://cwe.mitre.org/top25/archive/2023/2023_kev_list.html

- Code weaknesses
- Most prevalent and severe
- Analysis based on the exploited vulnerabilities



24 Deadly Sins of Software Security (1)

- How to write secure code
- It discusses most common design and coding errors and explains how to fix them

1. Sql Injection
2. Web Server–related Vulnerabilities (XSS, CSRF, and Response Splitting)
3. Web Client–related Vulnerabilities (XSS)
4. Use of magic Urls, predictable cookies, and hidden form fields
5. Buffer Overruns
6. Format String Problems
7. Integer Overflows
8. C++ Catastrophes
9. Catching Exceptions
10. Command Injection
11. Failure To Handle Errors Correctly
12. Information Leakage
13. Race Conditions
14. Poor Usability
15. Not updating easily
16. Executing code with too much privilege
17. Failure to protect stored data
18. The sins of mobile code
19. Use of weak password-based systems
20. Weak random numbers
21. Using the wrong cryptography
22. Failing to protect network traffic
23. Improper use of Pki, especially SSL
24. Trusting network name resolution

24 Deadly Sins of Software Security (2)

- How to write secure code
- It discusses most common design and coding errors and explains how to fix them
- Some are code-level (e.g., buffer overruns) others are more general and process-level

1. Sql Injection
2. Web Server–related Vulnerabilities (XSS, CSRF, and Response Splitting)
3. Web Client–related Vulnerabilities (XSS)
4. Use of magic Urls, predictable cookies, and hidden form fields
5. Buffer Overruns
6. Format String Problems
7. Integer Overflows
8. C++ Catastrophes
9. Catching Exceptions
10. Command Injection
11. Failure To Handle Errors Correctly
12. Information Leakage
13. Race Conditions
14. Poor Usability
15. Not updating easily
16. Executing code with too much privilege
17. Failure to protect stored data
18. The sins of mobile code
19. Use of weak password-based systems
20. Weak random numbers
21. Using the wrong cryptography
22. Failing to protect network traffic
23. Improper use of Pki, especially SSL
24. Trusting network name resolution

Classification of Software Security Errors (1)

1. Input Validation and Representation
2. API Abuse
3. Security Features
4. Time and State
5. Errors
6. Code Quality
7. Encapsulation
8. Environment errors

- It is a multi-layer organization of hundred of typical error
- For instance, the category “Input Validation and Representation” is divided in 26 subcategories of errors, e.g., Buffer Overflow, Command Injection, Cross-Site Scripting, Format String

K.Tsipenyuk, B.Chess, G. McGraw. “Seven Pernicious Kingdoms: A Taxonomy of Software Security Errors” IEEE Security and Privacy 2005 https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf

Classification of Software Security Errors (2)

1. Input Validation and Representation
2. API Abuse
3. Security Features
4. Time and State
5. Errors
6. Code Quality
7. Encapsulation
8. Environment errors

Buffer Overflow
Command Injection
Cross-Site Scripting
Format String
HTTP Response Splitting
Illegal Pointer Value
Integer Overflow
Log Forging
Path Manipulation
Process Control
Resource Injection
Setting Manipulation
SQL Injection
String Termination Error
Struts: Duplicate Validation Forms
Struts: Erroneous validate() Method
Struts: Form Bean Does Not Extend Validation Class
Struts: Form Field Without Validator
Struts: Plug-in Framework Not In Use
Struts: Unused Validation Form
Struts: Unvalidated Action Form
Struts: Validator Turned Off
Struts: Validator Without Form Field
Unsafe JNI
Unsafe Reflection
XML Validation

Code-level

Design-level

Technology-specific

- It is a multi-layer organization of hundred of typical error
- For instance, the category "Input Validation and Representation" is divided in 26 subcategories of errors, e.g., Buffer Overflow, Command Injection, Cross-Site Scripting, Format String

K.Tsipenyuk, B.Chess, G. McGraw. "Seven Pernicious Kingdoms: A Taxonomy of Software Security Errors" IEEE Security and Privacy 2005 https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf

Classification of Software Security Errors (3)

1. Input Validation and Representation
2. API Abuse
3. Security Features
4. Time and State
5. Errors
6. Code Quality
7. Encapsulation
8. Environment errors

Dangerous Function
Directory Restriction
Heap Inspection
J2EE Bad Practices: getConnection()
J2EE Bad Practices: Sockets
Often Misused: Authentication
Often Misused: Exception Handling
Often Misused: File System
Often Misused: Privilege Management
Often Misused: Strings
Unchecked Return Value

Code-level

Technology-specific

Design-level

- It is a multi-layer organization of hundred of typical error
- For instance, the category “API Abuse” is divided in 11 subcategories of errors

K.Tsipenyuk, B.Chess, G. McGraw. “Seven Pernicious Kingdoms: A Taxonomy of Software Security Errors” IEEE Security and Privacy 2005 https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf

Classification and groups of bugs / errors

Can we identify groups of "error" types?

Yes! Taxonomies and classifications exist, e.g.,

[1] CWE Top-25 and Top-10 most dangerous software weaknesses
[2] 24 Deadly Sins of Software Security: Programming Flaws and How to Fix Them
[3] Classification of Software Security Errors
[4] A Taxonomy of Software Security Defects for SST
[5] *Data Type Bugs Taxonomy: Integer Overflow, Juggling, and Pointer Arithmetics in Spotlight.
[6] *An empirical validation of a web fault taxonomy and its usage for web testing
[7] *Machine Learning-based Software Bug Classification through Mining Open-Source Repositories.
[8] *The Python/C API: Evolution, Usage Statistics, and Bug Patterns.
[9] OWASP Top-10

- ...

[1] https://cwe.mitre.org/top25/index.html
[2] M. Howard et al. 24 Deadly Sins of Software Security: Programming Flaws and How to Fix Them. McGraw-Hill Osborne Media, July 2009.
[3] K.Tsipenyuk et al. Seven Pernicious Kingdoms: A Taxonomy of Software Security Errors. IEEE Security and Privacy https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf
[4] Hui et al. A Taxonomy of Software Security Defects for SST. ICISS 2010
[5] I. Bojanova et al. Data Type Bugs Taxonomy: Integer Overflow, Juggling, and Pointer Arithmetics in Spotlight. STC 2022
[6] A.Marchetto et al. An empirical validation of a web fault taxonomy and its usage for web testing. JWE 2009
[7] J.P.Meher et al. Machine Learning-based Software Bug Classification through Mining Open Source Repositories. OCIT 2023
[8] The Python/C API: Evolution, Usage Statistics, and Bug Patterns. SANER 2020
[9] https://owasp.org/Top10/

* Not strictly related to security

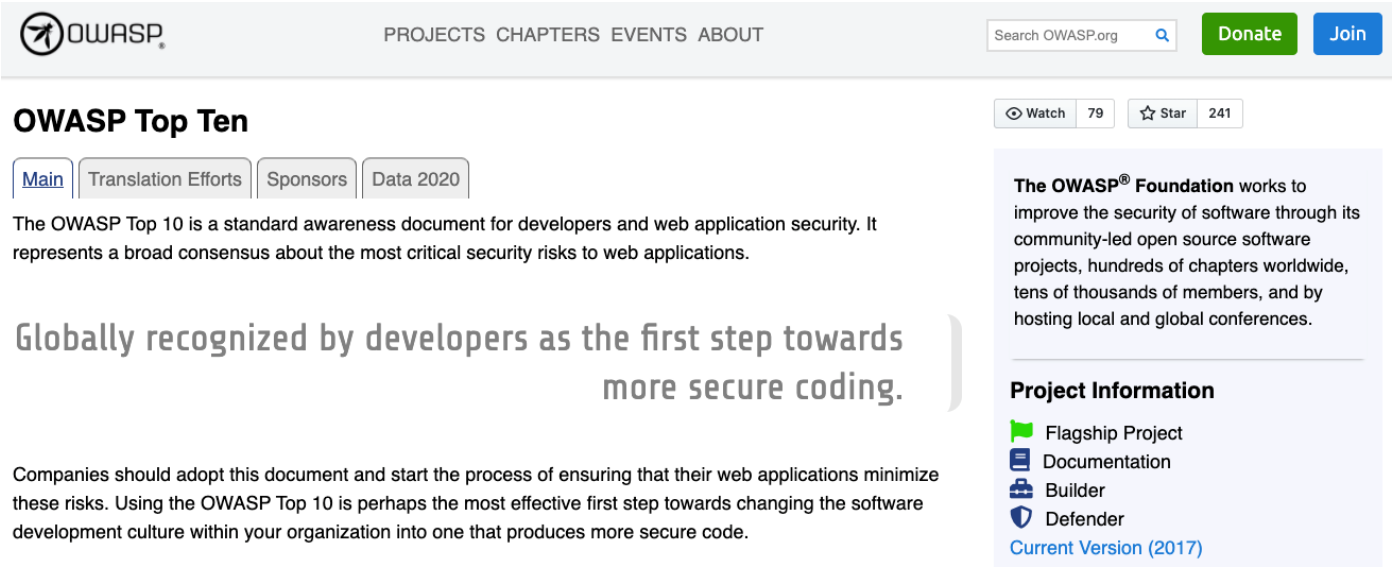
OWASP Top-10

OWASP

- It is an open-source project for the security of web applications
- It provides guides, guidelines, suggestions to develop and test secure applications

OWASP Top-10

- It lists the most critical security risks to web applications
- It represents a broad consensus about the most critical security risks to web applications.



https://owasp.org
https://owasp.org/www-project-top-ten/
https://owasp.org/Top10/

OWASP Top-10 of web application vulnerabilities

Id	Class	Note
A1	Broken Access Control	
A2	Cryptography Failures	
A3	Injection	
A4	Insecure Design	
A5	Security Misconfiguration	
A6	Vulnerable and Outdated Components	
A7	Identification and Authentication Failure	
A8	Software and Data Integrity Failure	
A9	Security Logging and Monitoring Failures	
A10	Server-Side Request Forgery (SSRF)	

https://owasp.org

OWASP Top-10 of web application vulnerabilities

Id	Class	Note
A1	Broken Access Control	It concerns access control issues - very frequent in web apps
A2	Cryptography Failures	It focuses on the root case for sensitive data expose issues
A3	Injection	It related to add unwanted code, - also frequent in web aps
A4	Insecure Design	It concerns design flaws
A5	Security Misconfiguration	It is related to various types of software configurations (e.g., apps and web server)
A6	Vulnerable and Outdated Components	It reflects the relevance of software reuse
A7	Identification and Authentication Failure	It shows that authentication is always an issue
A8	Software and Data Integrity Failure	It is somehow related to software update
A9	Security Logging and Monitoring Failures	It concerns aspects difficult to test (but useful for testing)
A10	Server-Side Request Forgery (SSRF)	It is not so frequent in data but relevant anyway

https://owasp.org

OWASP Top-10 evolution

OWASP Top-10 2021 → **Next edition planned for 2025**

2021
OWASP Top 10 (2021)
(A1) Broken Access Control
(A2) Cryptography Failures
(A3) Injection
(A4) Insecure Design
(A5) Security Misconfiguration
(A6) Vulnerable and Outdated Components
(A7) Identification and Authentication Failure
(A8) Software and Data Integrity Failure
(A9) Security Logging and Monitoring Failures
(A10) Server-Side Request Forgery (SSRF)

https://owasp.org/Top10/

OWASP Top-10 evolution

OWASP Top-10 2021 → **Next edition planned for 2025**

2013		2017			2021	
OWASP Top 10 (2013)	wrt 2017	OWASP Top 10 (2017)	wrt 2013	wrt 2021	OWASP Top 10 (2021)	wrt 2017
(A1) Injection	A1	(A1) Injection	A1	A3	(A1) Broken Access Control	A5
(A2) Broken Authentication and Session Management	A2	(A2) Broken Authentication	A2	A7	(A2) Cryptography Failures	A3
(A3) Cross-Site Scripting (XSS)	A7	(A3) Sensitive Data Exposure	A6	A2	(A3) Injection	A1+A7
(A4) Insecure Direct Object Reference	part of A5	(A4) XML External Entities (XXE)	NEW	part of A5	(A4) Insecure Design	NEW
(A5) Security Misconfiguration	A6	(A5) Broken Access Control	A4+A7	A1	(A5) Security Misconfiguration	A6+A4
(A6) Sensitive Data Exposure	A3	(A6) Security Misconfiguration	A5	part of A5	(A6) Vulnerable and Outdated Components	A9
(A7) Missing Function Level Access Control	part of A5	(A7) Cross-Site Scripting (XSS)	A3	part of A3	(A7) Identification and Authentication Failure	A2
(A8) Cross-Site Scripting Forgery(CSRF)	Drop	(A8) Insecure Deserialization	NEW	part of A8	(A8) Software and Data Integrity Failure	NEW +A8
(A9) Using Components with Known Vulnerabilities	A9	(A9) Using Components with Known Vulnerabilities	A9	part of A6	(A9) Security Logging and Monitoring Failures	A10
(A10) Unvalidated Redirect and Forwards	Drop	(A10) Insufficient logging and Monitoring	NEW	A9	(A10) Server-Side Request Forgery (SSRF)	NEW

https://owasp.org/Top10/

A01-2021:Broken Access Control

Description:

- Access control enforces policy such that **users cannot act outside of their intended permissions**. Failures typically lead to unauthorized information disclosure, modification, or destruction of all data or performing a business function outside the user's limits.
  - E.g., Bypassing access control checks by modifying the URL
  - E.g., Elevation of privilege.
  - E.g., Permitting viewing or editing someone else's account, by providing its unique identifier (insecure direct object references)

Prevention:

- Deny by default
- Access control is only effective in trusted server-side code or server-less API.
- Developers and QA staff should include functional access control unit and integration tests.

https://owasp.org/Top10/A01_2021-Broken_Access_Control/

A01-2021:Broken Access Control - *example*

Scenario #1: The application uses unverified data in a SQL call that is accessing account information:

```
PreparedStatement pstmt = connect.prepareStatement(query);
pstmt.setString(1, request.getParameter("acct"));   //acct is a “reference” to an account
ResultSet results = pstmt.executeQuery( );
```

- An attacker modifies the browser's 'acct' parameter to send whatever account number they want. If not correctly verified, the attacker can access any user's account.

  https://example.com/app/accountInfo?acct=notmyacct

Scenario #2: An attacker forces browses to target URL

https://example.com/app/getappInfo

https://example.com/app/admin_getappInfo

- If an unauthenticated user can access either page, it's a flaw. If a non-admin can access the admin page, this is a flaw.

A01-2021:Broken Access Control - *CWEs*

Examples of mapped CWEs

- CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
  https://cwe.mitre.org/data/definitions/22.html
- CWE-23: Relative Path Traversal
  https://cwe.mitre.org/data/definitions/23.html
- CWE-352 Cross-Site Request Forgery (CSRF)
  https://cwe.mitre.org/data/definitions/352.html
- CWE-601: URL Redirection to Untrusted Site ('Open Redirect')
  https://cwe.mitre.org/data/definitions/601.html
- CWE-566: Authorization Bypass Through User-Controlled SQL Primary Key
  https://cwe.mitre.org/data/definitions/566.html
- CWE-377: Insecure Temporary File
  https://cwe.mitre.org/data/definitions/377.html
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
  https://cwe.mitre.org/data/definitions/200.html

A02-2021:Cryptography Failures

Description:

- **Violation of the protection** needs for exchanged data. For example, passwords, credit card numbers, health records, personal information, and business secrets require extra protection, mainly if that data falls under privacy laws, e.g., EU's General Data Protection Regulation (GDPR), or regulations, e.g., financial data protection such as PCI Data Security Standard (PCI DSS).
  - E.g., Is any data transmitted in clear text?
  - E.g., Are any old or weak cryptographic algorithms or protocols used
  - E.g., Is the received server certificate and the trust chain properly validated?

Prevention:

- Proper encryption of all the sensitive data while storing and transmitting
- Proper setting of the used cryptographic mechanism

https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

A02-2021:Cryptography Failures - *example*

Example Attack Scenario

Scenario #1: An application encrypts credit card numbers in a database using automatic database encryption. However, this data is automatically decrypted when retrieved, allowing an SQL injection flaw to retrieve credit card numbers in clear text

Scenario #2:A site doesn't use TLS for all pages or supports weak encryption. An attacker monitors network traffic, downgrades connections from HTTPS to HTTP, intercepts requests, and steals the user's session cookie. The attacker then replays this cookie and hijacks the user's (authenticated) session, accessing or modifying the user's private data.

A02-2021:Cryptography Failures - *CWEs*

Examples of mapped CWEs:

- CWE-261: Weak Encoding for Password
  https://cwe.mitre.org/data/definitions/261.html
- CWE-319: Cleartext Transmission of Sensitive Information
  https://cwe.mitre.org/data/definitions/319.html
- CWE-321: Use of Hard-coded Cryptographic Key
  https://cwe.mitre.org/data/definitions/321.html

A03-2021: Injection

Description:

An application is vulnerable to attack when:

- User-supplied data is **not validated**, filtered, or **sanitized** by the application.
- Dynamic queries or non-parameterized calls **without context-aware escaping** are used directly in the interpreter.
- Hostile data is used within object-relational mapping (ORM) search **parameters to extract additional, sensitive records**.
- Hostile data is **directly used or concatenated**. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.

Prevention:

- Keeping data separate from commands and queries
- Use server-side input validation
- Use characters' escape
- Limit SQL queries

https://owasp.org/Top10/A03_2021-Injection/

A03-2021: Injection - *example*

Example Attack Scenario

Scenario #1: An application uses untrusted data in the construction of vulnerable SQL calls:

```
String query = "SELECT \* FROM accounts WHERE custID='" +
        request.getParameter("id") + "";
```

- The attacker modifies the ‘id’ parameter value in their browser to send: ‘ or ‘1’=’1. For example:

  http://example.com/app/accountView?id= ‘or’1’=‘1

  This changes the meaning of the query to return all the records from the accounts table

A03-2021: Injection - *CWEs*

Examples of mapped CWEs:

- CWE-20: Improper Input Validation
  https://cwe.mitre.org/data/definitions/20.html
- CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
  https://cwe.mitre.org/data/definitions/78.html
- CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
  https://cwe.mitre.org/data/definitions/79.html
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
  https://cwe.mitre.org/data/definitions/89.html
- CWE-94: Improper Control of Generation of Code ('Code Injection')
  https://cwe.mitre.org/data/definitions/94.html
- CWE-116: Improper Encoding or Escaping of Output
  https://cwe.mitre.org/data/definitions/116.html

A04-2021: Insecure Design

Description:

Insecure design concerns weaknesses related to "**missing or ineffective control design**."

- Requirements and Resource Management - collect and negotiate the business requirements for an application with the business, including the protection requirements concerning **confidentiality, integrity, availability**, and **authenticity** of all data assets and the expected business logic.
- Secure Design - a culture and methodology that constantly evaluates threats and ensures that code is robustly designed and tested to prevent known attack methods.
- Secure Development Lifecycle –security in the whole lifecycle

Prevention:

- Adopt a secure design methodology
- Proper implementation of security on the design stage, i.e., applying Threat Modeling, Tests

https://owasp.org/Top10/A04_2021-Insecure_Design/

A04-2021: Insecure Design - *example*

Example Attack Scenario

Scenario #1: A credential recovery workflow might include “questions and answers,” which is prohibited by NIST 800-63b, the OWASP ASVS, and the OWASP Top 10. Questions and answers cannot be trusted as evidence of identity as more than one person can know the answers, which is why they are prohibited. Such code should be removed and replaced with a more secure design.

Scenario #2: A cinema chain allows group booking discounts and has a maximum of fifteen attendees before requiring a deposit. Attackers could exploit this model flow and test if they could book six hundred seats and all cinemas at once in a few requests, causing a massive loss of income.

A04-2021: Insecure Design - *CWEs*

Examples of mapped CWEs:

- CWE-73: External Control of File Name or Path
  https://cwe.mitre.org/data/definitions/73.html
- CWE-256: Plaintext Storage of a Password
  https://cwe.mitre.org/data/definitions/256.html
- CWE-444: Inconsistent Interpretation of HTTP Requests ('HTTP Request/Response Smuggling’)
  https://cwe.mitre.org/data/definitions/444.html
- CWE-451: User Interface (UI) Misrepresentation of Critical Information
  https://cwe.mitre.org/data/definitions/451.html
- CWE-501: Trust Boundary Violation
  https://cwe.mitre.org/data/definitions/501.html

A05-2021: Security Misconfiguration

Description:

It refers to situations such as:

- Missing appropriate security hardening across any part of the application stack or **improperly configured permissions** on cloud services.
- **Unnecessary features** are enabled or installed (e.g., unnecessary ports, services, pages, accounts, or privileges).
- **Default accounts** and their passwords are still enabled and unchanged
- **Error handling** reveals stack traces or other overly informative error messages to users.
- The **software is out of date** or vulnerable

Prevention:

- Secure installation processes should be implemented
- A task to review and update of configurations

https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

A05-2021: Security Misconfiguration - *example*

Example Attack Scenario

Scenario #1: the application server comes with sample applications not removed from the production server. These sample applications have known security flaws attackers use to compromise the server. Suppose one of these applications is the admin console, and default accounts weren't changed. In that case, the attacker logs in with default passwords and takes over.

Scenario #2: the application server's configuration allows detailed error messages, e.g., stack traces, to be returned to users. This potentially exposes sensitive information or underlying flaws such as component versions that are known to be vulnerable.

A05-2021: Security Misconfiguration - *CWEs*

Examples of mapped CWEs:

- CWE-13: ASP.NET Misconfiguration: Password in Configuration File
  https://cwe.mitre.org/data/definitions/13.html
- CWE-15: External Control of System or Configuration Setting
  https://cwe.mitre.org/data/definitions/15.html
- CWE-537: Java Runtime Error Message Containing Sensitive Information
  https://cwe.mitre.org/data/definitions/537.html
- CWE-756: Missing Custom Error Page
  https://cwe.mitre.org/data/definitions/756.html

A06-2021:Vulnerable and Outdated Components

Description:

- The components (own and third-party ones) of a software can be **vulnerable, unsupported, and out of date**. This includes the OS, web/application server, database management system (DBMS), applications, APIs and all components, runtime environments, and libraries.

Prevention:

- Remove unused dependencies, unnecessary features, components, files, and documentation.
- Continuously inventory the versions of both client-side and server-side components (e.g., frameworks, libraries) and their dependencies
- Only obtain components from official sources over secure links.
- Continuous monitoring of (third-party) libraries and components

https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/

A06-2021:Vulnerable and Outdated Components - *example*

Example Attack Scenario

- Components typically run with the same privileges as the application itself, so flaws in any component can result in serious impact.
- Such flaws can be accidental (e.g., coding error) or intentional (e.g., a backdoor in a component).

An example: Log4jshell (2021)

- Log4j allows the user to change the default messages used to log,
- The user can also use Java Naming and Directory Interface (JNDI), an API for to discover and look up data and resources by means of a name
- By using JNDI however, attackers were able to execute (unwanted) remote code
- All software that used Log4j were exposed to this vulnerability

https://nvd.nist.gov/vuln/detail/CVE-2021-44228

A06-2021:Vulnerable and Outdated Components - *CWEs*

Examples of mapped CWEs:

- CWE-20: Improper input validation can lead to Vulnerable and Outdated Components
  https://cwe.mitre.org/data/definitions/20.html
- CWE-937: Using Components with Known Vulnerabilities
  https://cwe.mitre.org/data/definitions/937.html
- CWE-1035: Using Components with Known Vulnerabilities
  https://cwe.mitre.org/data/definitions/1035.html
- CWE-1104: Use of Unmaintained Third-Party Components
  https://cwe.mitre.org/data/definitions/1104.html

A07-2021: Identification and Authentication Failures

Description:

Authentication, confirmation of the user's identity, and **session management** is critical to protect against authentication-related attacks.

- E.g., Permits brute force or other automated attacks
- E.g., Permits default, weak, or well-known passwords, such as "Password1" or "admin/admin“
- E.g., Use plain text, encrypted, or weakly hashed passwords data stores
- E.g., Exposes session identifier in the URL.

Prevention:

- Implement proper identification and authentication processes. E.g., implement multi-factor authentication to prevent automated credential stuffing, brute force, and stolen credential reuse attacks.
- Do not permit weak password, e.g., align password length, complexity, and rotation policies
- Use a server-side, secure, built-in session manage

https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

A07-2021: Identification and Authentication Failures – *example*

Example Attack Scenario

Scenario #1: Most authentication attacks occur due to the continued use of passwords as a sole factor. Once considered, best practices, password rotation, and complexity requirements encourage users to use and reuse weak passwords. Organizations are recommended to stop these practices per NIST 800-63 and use multi-factor authentication.

Scenario #2: Application session timeouts aren't set correctly. A user uses a public computer to access an application. Instead of selecting "logout," the user simply closes the browser tab and walks away. An attacker uses the same browser an hour later, and the user is still authenticated.

A07-2021: Identification and Authentication Failures - *CWEs*

Example of CWEs

- CWE-259: Use of Hard-coded Password
  https://cwe.mitre.org/data/definitions/259.html
- CWE-306 Missing Authentication for Critical Function
  https://cwe.mitre.org/data/definitions/306.html
- CWE-384: Session Fixation
  https://cwe.mitre.org/data/definitions/384.html
- CWE-521: Weak Password Requirements
  https://cwe.mitre.org/data/definitions/521.html
- CWE-613: Insufficient Session Expiration
  https://cwe.mitre.org/data/definitions/613.html

A08-2021: Software and Data Integrity Failures

Description:

- It relates to code and infrastructure that does not protect **against integrity violations**.
  - An example is where an application relies upon **plugins, libraries, or modules from untrusted sources**, repositories, and content delivery networks (CDNs).
  - An insecure integration pipeline can introduce the potential for unauthorized access, malicious code, or system compromise.
- Many applications include **auto-update functionality**, where updates are done automatically without sufficient verification.
  - Attackers could potentially upload their own updates to be distributed and run on all installations.

Prevention:

- Ensure that consumed components come from trusted source.
- Use signatures (or similar mechanisms) to verify software and data

https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/

A08-2021: Software and Data Integrity Failures - *example*

Example Attack Scenario

Scenario #1: Update without signing - Many home routers, set-top boxes, and devices' firmware do not verify updates via signed firmware. Unsigned firmware is a growing target for attackers and is expected to only get worse. This is a major concern as many times there is no mechanism to remediate.

Scenario #2: Malicious update -The company that develops the software had secure build and update integrity processes. Still, these were able to be subverted, and for several months, the firm distributed a highly targeted malicious update to more than 18,000 organizations, of which around 100 were affected.

A08-2021: Software and Data Integrity Failures - *CWEs*

Examples of CWEs

- CWE-345: Insufficient Verification of Data Authenticity
  https://cwe.mitre.org/data/definitions/345.html
- CWE-494: Download of Code Without Integrity Check
  https://cwe.mitre.org/data/definitions/494.html
- CWE-502: Deserialization of Untrusted Data
  https://cwe.mitre.org/data/definitions/502.html
- CWE-830: Inclusion of Web Functionality from an Untrusted Source
  https://cwe.mitre.org/data/definitions/830.html

A09-2021: Security Logging and Monitoring Failures

Description:

- It relates to the logging and monitoring capability to detect security breaches
- You are vulnerable to **information leakage** by making logging and alerting events **visible** to a user or an attacker.
  - E.g., Auditable events, such as logins, failed logins, and high-value transactions, are not logged.
  - E.g., Logs of applications and APIs
  - E.g., The application cannot detect, escalate, or alert for active attacks in real-time or near real-time.

Prevention:

- Ensure proper logging and monitoring
- Establish or adopt an incident response and recovery plan

https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/

A09-2021: Security Logging and Monitoring Failures - *example*

Example Attack Scenario

Scenario #1: A children's health plan provider's website operator couldn't detect a breach due to a lack of monitoring and logging. An external party informed the health plan provider that an attacker had accessed and modified thousands of sensitive health records of more than 3.5 million children. A post-incident review found that the website developers had not addressed significant vulnerabilities. As there was no logging or monitoring of the system, the data breach could have been in progress since 2013, a period of more than seven years.

Scenario #2: An airline company had a data breach involving more than ten years' worth of personal data of millions of passengers, including passport and credit card data. The data breach occurred at a third-party cloud hosting provider, who notified the airline of the breach after some time.

A09-2021: Security Logging and Monitoring Failures - *CWEs*

Examples of mapped CWEs:

- CWE-117: Improper Output Neutralization for Logs
  https://cwe.mitre.org/data/definitions/117.html
- CWE-223: Omission of Security-relevant Information
  https://cwe.mitre.org/data/definitions/223.html
- CWE-532: Insertion of Sensitive Information into Log File
  https://cwe.mitre.org/data/definitions/532.html
- CWE-778: Insufficient Logging
  https://cwe.mitre.org/data/definitions/778.html

A10-2021: Server-Side Request Forgery (SSRF)

Description:

- SSRF flaws occur whenever a web application **is fetching a remote resource without validating the user-supplied URL**.
- The flow allows an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall, VPN, or another type of network access control list (ACL).

Prevention:

- Network layer: segment remote resource access functionality in separate networks to reduce the impact of SSRF, configure firewall policies to block all but essential traffic
- Application layer: sanitize and validate all client-supplied input data

https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/

A10-2021: Server-Side Request Forgery (SSRF) - *example*

Example Attack Scenario

Scenario #1: Port scan internal servers – If the network architecture is unsegmented, attackers can map out internal networks and determine if ports are open or closed on internal servers from connection results or elapsed time to connect or reject SSRF payload connections.

Scenario #2: Sensitive data exposure – Attackers can access local files such as or internal services to gain sensitive information such as file:///etc/passwd **and** http://localhost:28017/.

Scenario #3: Access metadata storage of cloud services – Most cloud providers have metadata storage such as http://169.254.169.254/. An attacker can read the metadata to gain sensitive information.

A10-2021: Server-Side Request Forgery (SSRF) - *CWEs*

Examples of mapped CWEs

- CWE-918: Server-Side Request Forgery (SSRF)
  - https://cwe.mitre.org/data/definitions/918.html